Problem Statement

I track my expenses and debt by hand in a FIRE spreadsheet, typing each expense into category columns. The FIRE part of the sheet is ignored because I don't know how to track it, so I have no view of my progress toward financial independence. Money sits uninvested in my PEA because nothing helps me decide, size, or schedule investments. My expense information already arrives automatically as emails at a dedicated Gmail address, but I still re-enter everything manually. Overall I lack a mature, continuous grasp of my finances: no budgets that push back, no insights into my spending patterns, no picture connecting monthly spending to long-term goals.

Solution

A private Discord bot backed by a database that becomes the single source of truth for my finances. It reads my finance inbox and turns emails into categorized transactions automatically, posting each to an audit feed where one tap fixes mistakes. It carries per-category budgets and proactively nudges me about my spending (fast food pace, bar visits, subscription creep) using full knowledge of my financial history. It computes net worth, savings rate, and FIRE progress from a two-minute monthly check-in. For my PEA it tracks positions, researches stocks on demand, enforces my own written guardrails, and schedules deployment of idle cash — while I alone execute every trade at my broker. Daily, weekly, and monthly reports plus on-demand questions give me the continuous financial awareness I'm missing.

User Stories

Expense ingestion

1. As a personal finance user, I want emails arriving at my finance inbox to be parsed into transactions automatically, so that routine expenses require zero typing.

2. As a personal finance user, I want bank transaction alert emails recognized and extracted (amount, merchant, date), so that card spending is captured in near real time.

3. As a personal finance user, I want merchant receipts and invoices of varied formats parsed, so that online orders and subscription charges are captured without per-merchant setup.

4. As a personal finance user, I want emails I manually forward to my finance address treated as expense sources, so that I can capture anything the automatic feeds miss.

5. As a PEA investor, I want my broker's Market Day summary emails recognized and processed, so that portfolio information flows in without effort.

6. As a personal finance user, I want each parsed transaction assigned to one of my categories automatically, so that my reports are meaningful without manual sorting.

7. As a personal finance user, I want deterministic merchant-to-category rules to take precedence over model-based categorization, so that a correction I make once holds forever.

8. As a personal finance user, I want duplicate detection across email sources, so that a receipt plus a bank alert for the same purchase doesn't count twice.

9. As a personal finance user, I want unparseable or ambiguous emails flagged to me instead of silently dropped, so that nothing falls through the cracks.

Audit feed & corrections

10. As a personal finance user, I want every committed transaction posted to an audit channel in Discord, so that I can watch my money move without opening anything.

11. As a personal finance user, I want a one-tap button to fix a transaction's category, so that correcting the system takes seconds.

12. As a personal finance user, I want buttons to edit or delete a transaction, so that errors never require leaving Discord.

13. As a personal finance user, I want my category corrections to automatically become rules, so that the system gets more accurate the more I use it.

Manual entry

14. As a personal finance user, I want to type a natural-language message like "15€ kebab" and have it logged correctly, so that cash spending has near-zero friction.

15. As a personal finance user, I want a structured slash command with category autocomplete, so that I can enter expenses precisely when I prefer.

Categories & budgets

16. As a personal finance user, I want my existing categories preserved as subcategories under a small set of groups, so that my mental model of spending stays intact while reports get readable.

17. As a budget-conscious user, I want to set a monthly budget on any category or group, so that my intentions are encoded, not just remembered.

18. As a budget-conscious user, I want a guided flow to set my initial budgets, so that my intentions are encoded from day one.

19. As a budget-conscious user, I want an alert when a category crosses a configurable share of its budget, so that I can adjust before the month is lost.

20. As a budget-conscious user, I want an alert when a budget is exceeded, so that overspending is never a surprise at month end.

Nudges & insights

21. As a budget-conscious user, I want proactive nudges about my spending pace in discretionary categories (fast food, bar, eating out), so that I course-correct mid-month rather than regret at month end.

22. As a personal finance user, I want the system to use my complete financial history when generating insights, so that observations are grounded in my actual patterns rather than generic advice.

23. As a personal finance user, I want to be told about spending trends (a category creeping up over months), so that slow drifts become visible.

24. As a personal finance user, I want subscription-creep detection driven by my recurring-charge registry, so that forgotten subscriptions get cancelled.

25. As a personal finance user, I want unusual transactions flagged (abnormal size or merchant), so that mistakes and fraud surface immediately.

26. As a budget-conscious user, I want nudges to be specific and quantified ("3 bar visits this week, €48 — on pace to hit the monthly budget by the 12th"), so that they are actionable, not naggy.

Reports & queries

27. As a personal finance user, I want an evening summary of the day's transactions and month-to-date total, so that daily awareness is effortless.

28. As a personal finance user, I want a weekly pulse digest of spending, notable transactions, and portfolio moves, so that I start each week oriented.

29. As a personal finance user, I want a monthly report covering category spend vs budget, savings rate, net worth, portfolio performance, and FIRE progress, so that I have one anchor ritual for my finances.

30. As a personal finance user, I want to mute or change the cadence of any report independently, so that the bot's volume matches my appetite.

31. As a personal finance user, I want to ask free-form questions ("how much on groceries this month?", "biggest expenses this quarter?") and get answers computed from my data, so that any question has an immediate answer.

Net worth & FIRE

32. As a FIRE aspirant, I want a short monthly check-in asking for each account balance and a one-tap confirmation of the month's income, so that net-worth tracking and the savings-rate denominator cost two minutes a month.

33. As a FIRE aspirant, I want my net worth computed and trended over time, including debts as negatives, so that I see the true trajectory.

34. As a FIRE aspirant, I want my monthly savings rate computed from income and expense-flow spending — transfers to savings, PEA, and debt repayment count as saving, never as spending — so that the single most important FIRE number is always current.

35. As a FIRE aspirant, I want my FI number derived from rolling twelve-month expense-flow spending and my withdrawal-rate assumption, so that the target reflects how I actually live.

36. As a FIRE aspirant, I want my progress toward the FI number shown as a percentage, so that FIRE feels like a real, moving goal.

37. As a FIRE aspirant, I want to recalibrate my assumptions (withdrawal rate, expected return) through a guided interactive session, so that the placeholders in my old sheet are replaced with reasoned values.

38. As a FIRE aspirant, I want a one-time settings-seeded bootstrap — opening debt balances entered by command, a net-worth baseline from my first check-in, and a baseline annual-expenses assumption — so that FIRE metrics are meaningful from day one without importing my old spreadsheet.

Debt

39. As a personal finance user, I want each debt stored with balance, rate, and minimum payment, so that my liabilities are as visible as my assets.

40. As a personal finance user, I want debt payments detected from my transaction stream, so that balances stay current automatically.

41. As a personal finance user, I want a projected payoff date per debt, so that the end is always in sight.

42. As a FIRE aspirant, I want an interest-vs-invest comparison for extra payments, so that I put marginal euros where they work hardest.

PEA portfolio & advisor

43. As a PEA investor, I want my trades auto-logged from broker execution-confirmation emails, so that my positions stay true with zero effort.

44. As a PEA investor, I want a command to log trades manually, so that missed confirmations never corrupt the portfolio view.

45. As a PEA investor, I want my positions valued with market data, so that portfolio worth is always current.

46. As a PEA investor, I want portfolio performance included in my reports, so that investment results live beside my spending.

47. As a PEA investor, I want an alert when a position falls a configurable percentage from entry, so that losses are visible early — alert only, never auto-sell.

48. As a PEA investor, I want a warning when a planned buy would push a single position or sector past my cap, so that conviction can't quietly become concentration.

49. As a PEA investor, I want my idle cash deployed on a fixed calendar with reminders, so that I invest on schedule instead of timing the market.

50. As a PEA investor, I want new positions to require a written thesis and a cooling-off period before the bot marks them approved, so that impulse buys die in the waiting room.

51. As a PEA investor, I want on-demand research on a ticker (fundamentals, news synthesis, thesis check against my guardrails), so that decisions are informed without hours of reading.

52. As a PEA investor, I want scheduled scans of my held positions for material news or fundamental changes, so that I hear about problems before my monthly review.

53. As a PEA investor, I want every recommendation framed as decision support that I execute manually at my broker, so that no system ever moves my money.

System & administration

54. As the system owner, I want all credentials stored only on my server, so that my financial access never lives in third-party hands.

55. As the system owner, I want an admin-only command to simulate the passage of time, so that scheduled behavior can be tested end-to-end on demand.

56. As the system owner, I want to export my data to a spreadsheet at any time, so that I'm never locked in.

Transfers

57. As a personal finance user, I want every transaction to carry a flow — expense, income, or transfer — so that moving my own money is never counted as spending.

58. As a personal finance user, I want outgoing virement emails recognized as transfers with the beneficiary resolved to one of my accounts or debts by deterministic rules, so that savings moves and repayments are captured without typing.

59. As a personal finance user, I want to log inbound transfers manually (money returning to checking), so that round-trips through my fun-money account stay invisible to income and expense math.

60. As a personal finance user, I want transfer audit posts with fix-counterpart and fix-flow buttons, so that a mis-read virement is a one-tap correction that becomes a rule.

61. As a FIRE aspirant, I want account balances to come only from my monthly check-in while transfers remain flow records (debt balances alone decrement on detected payments), so that the books never silently drift from reality.

62. As a personal finance user, I want a monthly reconciliation view comparing transfers sent against balance growth per account, so that interest and untracked activity are visible, not absorbed.

Recurring charges & committed money

63. As a budget-conscious user, I want a registry of my recurring charges — bills, subscriptions, and standing transfers — so that what fires automatically each month is encoded, not remembered.

64. As a personal finance user, I want detected recurrences proposed to me for one-tap confirmation, plus a command to seed known ones, so that the registry starts complete and stays honest.

65. As a personal finance user, I want incoming transactions matched to registry entries deterministically, so that price increases and missed charges are flagged reliably.

66. As a budget-conscious user, I want my month presented as committed consumption, committed saving, and discretionary shares of payroll, so that "pay myself first" is a visible fact.

67. As a FIRE aspirant, I want my payroll stored as a setting and confirmed at the monthly check-in, so that the savings rate and committed shares always have a true denominator.

Implementation Decisions

- Single-process application: typescript with discordjs, hosted on a small VPS, timezone Europe/Paris, currency EUR.

- DockerFile / Docker Compose to rapidly and easily deploy the bot anywhere.

- PostgreSQL is the single source of truth. Core entities: accounts, transactions, categories (two-level: group → subcategory), category rules, beneficiary rules, budgets, recurring charges, debts, positions, trades, theses, insight/nudge log, settings, balance snapshots.

- Hexagonal architecture: the application core is driven exclusively by three inbound events — an email received, a Discord interaction, a clock tick — and acts on the world exclusively through outbound ports: Discord sends, LLM calls, market-data fetches. No business logic in adapters.

- Gmail ingestion (ADR 0008): the mailbox is a shared personal inbox, not a dedicated address — the owner's Gmail filters (plus-addressing and bank senders) apply a finance label, and Talis polls exactly the mail carrying that label (`GMAIL_FINANCE_LABEL`, default `Finances`) via the Gmail API with an OAuth refresh token every few minutes. The label is the contract: new senders are captured by editing filters or forwarding to the +finance address; sub-labels are ignored; ingestion is live-only (no historical backfill); the production token is read-only (`gmail.readonly`); processed message IDs recorded for idempotency.

- Agent layer (ADR 0003): Mastra, embedded as a library inside the hexagonal boundary. Hand-written adapters keep owning all I/O; Mastra provides the intelligence layer the

core calls through its ports — Agents (parser, insight/nudge, advisor/research), Workflows as short-lived pipelines (email → transaction, thesis flow, recalibration), and Memory. Mastra's server/deployer/playground are not part of the production system.

- Time ownership (ADR 0004): the core tick engine owns all scheduling via its simulatable clock. Cooling-off expiry, DCA calendar, and report cadence are Postgres rows evaluated on each tick; Mastra workflows never sleep or suspend across wall-clock time, so `/simulate` fast-forwards every time-dependent behavior.

- Memory (ADR 0005): working + observational memory via `@mastra/pg` in the application's Postgres, scoped to conversational agents (advisor, Q&A, recalibration). Memory holds preferences, reactions, and reasoning context — never financial facts; every euro figure comes from a deterministic tool. The nudge-dedup log remains a plain table; observational memory only tunes nudge tone and relevance. Mastra Workspace is enabled only for the research agent (filesystem tools for notes, thesis drafts, scan artifacts; shell execution disabled).

- Agent data access (ADR 0006): curated read-only aggregate tools (spend by category/period, budget status, net-worth series, portfolio value) plus a fallback `sql_readonly` tool under a SELECT-only Postgres role with statement timeout and row cap. Write paths are never agent tools — all mutations are core code triggered by workflows and Discord interactions.

- Models: Haiku-class (claude-haiku-4-5) for email classification and extraction; Sonnet-class (claude-sonnet-5) for insights, Q&A, and advisor/research. Model ids are configuration in the settings table.

- Parsing: the parser agent classifies the email and extracts amount, merchant, date, and category with structured output; a deterministic merchant→category rule table overrides the model. Corrections from audit-feed buttons upsert rules.

- Transactions auto-commit on parse; the audit feed is post-hoc correction, not pre-commit approval.

- Transaction flow (ADR 0012): every transaction carries a flow — expense | income | transfer — enforced at the schema level. Only expense rows feed budgets, category aggregates, nudges, and the rolling 12-month expenses behind the FI number; cross-source dedup (ADR 0010) is scoped to expense rows. Transfers are category-less, slim directional records: direction out is parsed from the bank's outgoing-operation emails (a dedicated parser classification), direction in is manual (no email exists); the counterpart — an account from the single accounts registry (the same rows the check-in snapshots) or a debt — is resolved by a beneficiary→counterpart rule table mirroring merchant rules, optional except for debt payments, which decrement the debt. Balances are check-in-authoritative; transfers never mutate account balances;

reconciliation is a report. Transfer audit posts carry fix-counterpart and fix-flow buttons with message-command mirrors (ADR 0009).

- Recurring-charge registry (ADR 0013): a registry of expectations over ordinary transactions — merchant/beneficiary pattern, expected amount, cadence, kind (bill | subscription | transfer), active — never a second ledger. Entries enter by owner-confirmed detection proposals or /recurring seeding; matching is deterministic (pattern + amount tolerance + cadence window), never an LLM call. The registry powers the committed split (committed consumption / committed saving / discretionary vs payroll) in the monthly report, subscription-creep detection, price-change and missed-charge alerts. Monthly payroll is a setting confirmed at the check-in; confirmation writes a flow=income row.

- Category taxonomy: the existing 17 spreadsheet categories become subcategories under approximately 6 groups; budgets attachable at either level; initial budgets set manually through a guided setup flow.

- Insights/nudge engine runs on transaction-commit and on scheduled ticks, computing deterministic aggregates (pace vs budget, trends, recurrence) and using the LLM port over those aggregates for pattern-spotting and phrasing. Every nudge is logged to prevent repetition.

- FIRE metrics: FI number = rolling 12-month expenses × (1 / withdrawal rate); until 12 real months of data exist, a baseline-annual-expenses assumption from settings substitutes, blending proportionally as real months accrue. Savings rate from monthly income vs outflow; assumptions stored in settings and editable via a guided recalibration flow.

- Market data via free tiers (delayed quotes and history from a Yahoo-Finance-derived source; free fundamentals APIs; news via RSS and web search) behind a single market-data port so paid providers can substitute later.

- Guardrail parameters (position cap %, sector cap %, drawdown %, DCA schedule, cooling-off duration) live in settings, enforced as warnings in advisor output — the system has no order-execution capability by design.

- Bootstrap, no migration: the legacy spreadsheet is never imported. Opening debt balances are entered by command; the first monthly check-in establishes the net-worth baseline; baseline annual expenses live in the FIRE assumptions. History-dependent features (trends, savings-rate history) report honestly from live data only, ramping up as months accumulate.

- Reports are generated from ticks; each report type has independent enable/cadence settings.

- Admin-only simulate-time command advances the scheduler's clock for testing; guarded by owner ID.

- Secrets (Discord token, Gmail OAuth, LLM key) provided via environment/secret file on the VPS only.

Testing Decisions

- A good test exercises only externally observable behavior: what the bot says in Discord and, where unavoidable, exported data. Tests never inspect internal modules, call internal functions, or mock internal collaborators. Internal refactors must never break a test.

- The seam is the system's real external boundary — the highest possible: end-to-end tests run against a real Discord test server and a real test Gmail inbox, with real LLM calls. A test-harness Discord account sends commands and reads the bot's replies; the harness sends real emails to the test inbox and waits for the resulting audit-feed messages.

- Time-dependent behavior (reports, DCA reminders, cooling-off expiry) is tested via the admin simulate-time command — the single test affordance in the production surface.

- Determinism strategy: LLM-dependent assertions check structured outcomes (amount, category, presence of buttons), not exact prose; email fixtures are anonymized real samples covering each source type.

- Suite layout: a small smoke subset (one path per slice) runs on every commit; the full suite runs nightly, serially, respecting Discord and Gmail rate limits.

- Prior art: none — this is a greenfield project; this E2E harness is the founding test pattern and each vertical slice must ship with its harness coverage.

Out of Scope

- Any execution of trades, transfers, or payments — the system never moves money, under any configuration.

- Bank account aggregation APIs (Powens/Bridge class) and credential-based bank scraping.

- Real-time intraday market data or anything supporting day trading.

- Web or mobile UI beyond Discord.

- Multi-user support; the system serves exactly one owner.

- Tax computation or official reporting (French tax declaration support may be a future PRD).

- Automatic portfolio rebalancing logic.

- Any import of, or sync with, the legacy spreadsheet (on-demand export only).

Further Notes

- Vocabulary follows the project design document: transaction, audit feed, check-in, nudge, guardrail, thesis, cooling-off, FI number, slice.

- Delivery is by vertical slices, each independently useful end-to-end: (1) one email type to audit feed with manual entry; (2) full ingestion, corrections-to-rules, budgets, reports, nudges; (3) net worth, FIRE metrics, debt, recalibration; (4) positions, valuation, drawdown alerts, DCA calendar; (5) research, thesis workflow, guardrail checks, scheduled scans.

- The advisor's output is decision support, not financial advice; the recalibration session presents evidence and trade-offs, and the owner sets the values.

- Prerequisites before slice 1: VPS, Discord server + bot application, Gmail OAuth credentials for the finance inbox, LLM API key.

- Revision 2026-07-04: Mastra decisions concretized (ADRs 0003–0006 in-repo); spreadsheet migration dropped in favor of settings-seeded bootstrap (stories #18 and #38 rewritten, Slice 6 issue re-scoped).

- Revision 2026-07-10: transactions gain a flow discriminator separating transfers (savings, PEA funding, debt repayments) from spending, and a recurring-charge registry powers a committed/discretionary view (ADRs 0012–0013). Stories 57–67 added; stories 24, 32, 34, 35 amended. Two slices inserted in delivery order: Transfers (#31, after slice 6, before budgets) and Recurring-charge registry (#32, after budgets); slice 10 gains registry proposals; slice 11's check-in gains the income confirmation.

- Revision 2026-07-05: the "dedicated Gmail address" assumption replaced with the owner's real mailbox — a shared personal inbox where filters label finance mail. Ingestion is label-gated (ADR 0008): the label is the contract, sub-labels ignored, live-only, production Gmail token read-only.